First Day at Office — Backend Dev Intern Survival Guide

What to Expect

Your first day is mostly setup and orientation. You will:

1. Get access to tools and repositories
2. Set up your development environment
3. Read documentation
4. Maybe fix a small bug or do a starter task

Nobody expects you to ship production code on day one. Breathe.

Before You Walk In

On Your Machine (Do Tonight)

```
# Verify these are installed and working
python3 --version       # 3.11+
git --version           # 2.x+
code --version          # VS Code
docker --version        # Docker Desktop running
pip3 --version
curl --version
```

Accounts You Might Need

- [ ] GitHub or GitLab (company org — they'll invite you)
- [ ] Slack or Teams (communication)
- [ ] Jira, Linear, or Asana (task tracking)
- [ ] 1Password, Vault, or similar (secrets manager)
- [ ] Cloud console (AWS, GCP, Azure) — maybe not day one
- [ ] Postman (team workspace) — they might have shared collections

---

Hour by Hour: What Typically Happens

Hour 1-2: Onboarding and Access

You'll meet your team lead or buddy. They'll:

- Add you to Slack channels
- Give you repo access
- Share credentials / environment variables

- Point you to documentation

Your job: Write everything down. Open a notes file.

```
# Keep a running log
cat > ~/work-notes.md
# Day 1 Notes
# Team lead: [name]
# Slack channels: #backend, #dev-general
# Main repo: https://github.com/company/[repo]
# Dev database: [they'll tell you]
# API docs: [URL]
```

`Ctrl+D` to save. Update this throughout the day.

Hour 2-3: Clone and Set Up the Project

They'll give you a repo URL. Here's the typical flow:

```
# Step 1: Clone the repo
git clone https://github.com/company/backend-api.git
cd backend-api

# Step 2: READ THE README FIRST
cat README.md
# or open it in VS Code
code README.md
```

The README should tell you how to set up. If it doesn't, ask. A typical Python/FastAPI setup:

```
# Step 3: Create virtual environment
python3 -m venv .venv
source .venv/bin/activate

# Step 4: Install dependencies
pip install -r requirements.txt
# or
pip install -r requirements-dev.txt

# Step 5: Set up environment variables
cp .env.example .env
# Then edit .env with the values they give you
nano .env
# or
code .env
```

What goes in `.env`:

```
DATABASE_URL=postgresql://user:password@localhost:5432/companydb
SECRET_KEY=some-key-they-give-you
REDIS_URL=redis://localhost:6379
API_KEY=whatever-they-share
```

Ask your team lead for these values. Never guess.

```
# Step 6: Set up the database (if local)
# They might use Docker for this:
docker compose up -d db

# Or you might need to run migrations:
alembic upgrade head

# Step 7: Start the server
uvicorn app.main:app --reload
# or whatever their start command is (check README or
package.json/Makefile)
```

```
# Step 8: Verify it works
curl http://localhost:8000/health
# Should return something like {"status": "healthy"}

# Step 9: Open the API docs
open http://localhost:8000/docs
# This shows you every endpoint in the API
```

If any step fails, **don't spend more than 15 minutes stuck.** Ask your team.

Hour 3-4: Explore the Codebase

Before writing any code, understand the project structure.

```
# See the overall structure
ls -la
tree -L 2        # if tree is installed, otherwise:
find . -type f -name "*.py" | head -30

# Key files to read first:
cat app/main.py              # App entry point — what routes exist?
ls app/routes/               # All API endpoints
ls app/models/               # Data models
ls app/services/             # Business logic
```

```
cat requirements.txt         # What libraries are used?
cat docker-compose.yml       # What services does the app depend on?
cat Makefile                 # Shortcut commands (if exists)
```

Questions to answer by reading the code:

1. What framework? (FastAPI, Flask, Django?)
2. What database? (PostgreSQL, MySQL, MongoDB?)
3. What ORM? (SQLAlchemy, Tortoise, raw SQL?)
4. How is auth done? (JWT, sessions, API keys?)
5. How are tests run? (`pytest`, `make test`, etc.)
6. How is the app deployed? (Docker, Kubernetes, Heroku?)

Hour 4-5: Interact with the API

Use the tools you know to explore the running API.

Using the Swagger Docs

```
Open http://localhost:8000/docs in your browser
Click any endpoint → "Try it out" → "Execute"
```

This is the fastest way to understand what the API does.

Using curl

```
# GET — read data
curl http://localhost:8000/api/v1/users
curl http://localhost:8000/api/v1/users/1

# GET with query parameters
curl "http://localhost:8000/api/v1/users?page=1&limit=10"

# POST — create data
curl -X POST http://localhost:8000/api/v1/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Test User", "email": "test@example.com"}'

# With authentication (if required)
TOKEN="the-token-you-got-from-login"
curl http://localhost:8000/api/v1/users/me \
  -H "Authorization: Bearer $TOKEN"

# See full request/response details
curl -v http://localhost:8000/api/v1/users
```

Using HTTPie (cleaner syntax)

```
# GET
http GET localhost:8000/api/v1/users

# POST
http POST localhost:8000/api/v1/users \
  name="Test" email="test@example.com"

# With auth
http GET localhost:8000/api/v1/users/me \
  Authorization:"Bearer $TOKEN"
```

Using Postman

1. Import the OpenAPI spec: `http://localhost:8000/openapi.json`
2. Postman auto-generates a collection of all endpoints
3. Click and send — no typing needed

Hour 5-6: Git Workflow Practice

Before you touch real code, make sure you know the team's Git workflow.

Ask these questions:

- What branch do I branch off from? (`main`? `develop`?)
- What's the branch naming convention? (`feature/`, `fix/`, ticket numbers?)
- How many reviewers needed for a PR?
- Any CI checks that must pass?

Your first branch (when you get a task)

```
# Always start from the latest main
git checkout main
git pull origin main

# Create your branch
git checkout -b feature/your-first-task

# Do your work...
# ...

# Check what you changed
git status
git diff

# Stage and commit
git add app/routes/users.py
```

```
git commit -m "Add email validation to user registration"

# Push your branch
git push -u origin feature/your-first-task

# Then open a Pull Request on GitHub/GitLab
```

Essential Commands You'll Use Every Day

Terminal Navigation

```
pwd                     # Where am I?
ls -la                  # What's here?
cd app/routes           # Go to routes directory
cd ../..                # Go up two levels
cd -                    # Go back to previous directory
code .                  # Open current directory in VS Code
```

Git (You'll Run These 20+ Times a Day)

```
git status              # What changed?
git diff                # What exactly changed?
git add <file>          # Stage a file
git commit -m "msg"     # Commit
git push                # Push to remote
git pull origin main    # Get latest
git checkout -b name    # New branch
git checkout main       # Switch to main
git log --oneline -10   # Recent history
git stash               # Temporarily save uncommitted changes
git stash pop           # Bring them back
```

Server Management

```
# Start the dev server
uvicorn app.main:app --reload

# Check if something is running on a port
lsof -i :8000

# Kill a process on a port
kill $(lsof -t -i :8000)

# Docker
docker compose up -d          # Start services
```

```
docker compose down            # Stop services
docker compose logs -f app     # Watch logs
docker compose exec db psql -U postgres  # Access database
```

Reading and Searching Code

```
# Search for something in the codebase
grep -r "def create_user" app/
grep -r "TODO" app/
grep -rn "DATABASE_URL" .      # -n shows line numbers

# Find files
find . -name "*.py" -path "*/routes/*"

# Read a file quickly
cat app/main.py
head -50 app/main.py           # First 50 lines
tail -20 app/services/auth.py  # Last 20 lines
```

Things That Will Go Wrong (And How to Fix Them)

"Module not found" error

```
# Did you activate your virtual environment?
source .venv/bin/activate
# Check:
which python    # Should point to .venv/bin/python
```

"Address already in use" (port 8000)

```
# Something else is using the port
lsof -i :8000
kill $(lsof -t -i :8000)
# Try again
uvicorn app.main:app --reload
```

"Permission denied"

```
# Make a script executable
chmod +x script.sh

# Or use sudo (only if you understand why)
sudo <command>
```

Database connection failed

```
# Is the database running?
docker compose ps

# Start it if it's not
docker compose up -d db

# Check if you can connect
psql -U postgres -h localhost -d companydb
```

Git merge conflict

```
# Don't panic. Open the file, look for:
# <<<<<<< HEAD
# (your changes)
# =======
# (their changes)
# >>>>>>>

# Pick the right version, remove the markers, then:
git add <file>
git commit -m "Resolve merge conflict in <file>"
```

"I broke something and want to undo"

```
# Undo uncommitted changes to a specific file
git checkout -- app/routes/users.py

# Undo ALL uncommitted changes (CAREFUL)
git checkout -- .

# Undo last commit but keep the changes
git reset --soft HEAD~1
```

What to Say When You're Stuck

Don't say: "It doesn't work."

✅ but got [error message]. I tried [what you attempted]. Can you help me figure out what I'm missing?"

This shows you've tried and gives them enough context to help fast.

End of Day Checklist

```
[ ] Dev environment runs (server starts, can hit endpoints)
[ ] Can access the repo and push branches
[ ] Joined relevant Slack channels
[ ] Read the README and key parts of the codebase
[ ] Know who to ask for help
[ ] Noted down credentials, URLs, and setup steps
[ ] Know the team's Git workflow
[ ] First task assigned (or know when it will be)
```

Quick Reference Card

Print this or keep it open on your second monitor.

```
NAVIGATE          BUILD/RUN              GIT
pwd               uvicorn ... --reload   git status
ls -la            docker compose up -d   git pull origin main
cd <dir>          docker compose down    git checkout -b feature/x
cd ..             pytest                 git add <file>
cd -              pip install -r req.txt git commit -m "msg"
                                         git push -u origin feature/x
DEBUG             SEARCH                 git stash / git stash pop
curl -v <url>     grep -rn "text" .
lsof -i :8000     find . -name "*.py"   ASK FOR HELP
docker logs app   cat <file>            After 15-30 min of being stuck
cat .env          code .                Not after 3 hours of silence
```

You're more prepared than you think. Ship it.